CIS 613:
Unit Testing Tools

1

Unit Testing

- Unit testing tools originated with SUnit (for Smalltalk) developed by Kent Beck, the 'creator' of *Extreme Programming*
- Spawned a whole plethora of 'xUnit' frameworks*:
  - JUnit – For Java programs
  - CUnit – For C programs
  - CppUnit – For C++ programs
  - NUnit – For all .Net programs
  - PyUnit (unittest) – For Python programs
  - …

What is Unit Testing?

* https://en.wikipedia.org/wiki/List_of_unit_testing_frameworks

2

JUnit: Assertions

- assert[Type of Assertion](expected, actual)
- assert[Type of Assertion](message, expected, actual)
- Full set can be found here: https://junit.org/junit4/javadoc/4.8/org/junit/Assert.html

3

JUnit: Assertions

assertEquals(double expected, double actual) is depreciated. Why?

A. You should include a message.
B. The data type double shouldn't be used in Java.
C. You can't compare two doubles for equality.
D. It was changed to use the 'float' data type.

4

ElcEmma

Provides code coverage visualizations and metrics for Java programs.

Launch types supported*:
- Local Java Application
- Eclipse/RCP Application
- Equinox OSGi Framework
- JUnit Test
- TestNG Test
- JUnit Plug-In Test
- JUnit RAP Test
- SWTBot Test
- Scala Application

Which ones would you use for unit and system/integration testing?

* http://www.eclemma.org

5

ElcEmma

Provides code coverage visualizations and metrics for Java programs.

Launch types supported*:
- Local Java Application
- Eclipse/RCP Application
- Equinox OSGi Framework
- JUnit Test
- TestNG Test
- JUnit Plug-In Test
- JUnit RAP Test
- SWTBot Test
- Scala Application

Which ones would you use for unit and system/integration testing?

* http://www.eclemma.org

6

EclEmma

Analysis*:

- **Coverage overview**: The Coverage view lists coverage summaries for your Java projects, allowing drill-down to method level.
- **Source highlighting**: The result of a coverage session is also directly visible in the Java source editors. A customizable color code highlights fully, partly and not covered lines. This works for your own source code as well as for source attached to instrumented external libraries.

* http://www.eclemma.org

7

EclEmma

Analysis*:

- **Different counters**: Select whether instructions, branches, lines, methods, types or cyclomatic complexity should be summarized.
- **Multiple coverage sessions**: Switching between coverage data from multiple sessions is possible.
- **Merge Sessions**: If multiple different test runs should be considered for analysis coverage sessions can easily be merged.

Why would we care about merging coverage sessions?

* http://www.eclemma.org

8

ElcEmma

Report Generation / Export & Import*:

- **Execution data import**: A wizard allows to import JaCoCo *.exec execution data files from external launches.
- **Coverage report export**: Coverage data can be exported in HTML, XML or CSV format or as JaCoCo execution data files (*.exec).

* http://www.eclemma.org

9

ElcEmma

Demo

10

What about Python?

11

‘coverage’ Package

Allows you to:

- Record coverage for a variety of launches, including program execution *and* test execution.
  Command: coverage run .\[pythonfile].py
- Append coverage to existing coverage records.
  Command: coverage run **-a** .\[pythonfile].py
- Report coverage numbers for the recorded coverage levels.
  Command: coverage report –m
- Report coverage in HTML format for the recorded coverage levels.
  Command: coverage html

* https://coverage.readthedocs.io/en/latest/cmd.html

12

‘coverage’ Package

Demo

13

Code Coverage

Is code coverage a good measure of how much to test?

14

Is Code Coverage Enough?

```
public static int weightedAverage(int weight1,
                                  int average1,
                                  int weight2,
                                  int average2) {

  if(weight1<0 || average1<0 ||
    weight2<0 || average2<0)
    throw new IllegalArgumentException();

  if(weight1>500 || average1>500 ||
    weight2>500 || average2>500)
    throw new IllegalArgumentException();

  return ((weight1 * average1) + (weight2 * average2)) / (weight1 + weight2);
}
```

Any Errors in this Code?

15

Basis Path Testing

Then basis path testing must be a better method, right?

16

Reverse String Testing: In Class Exercise

```
1. String reverseString(String string) {
2.     String reverse = "";
3.     if(string != null) {
4.         for(int i = string.length()-1; i >= 0; i--) {
5.             reverse += string.charAt(i);
6.         }
7.     }
8.     return reverse;
9. }
```

17

Reverse String Testing: In Class Exercise

```
1. String reverseString(String string) {
2.     String reverse = "";
3.     if(string != null) {
4.         if(string.length() > 0) {
5.             for(int i = string.length()-1; i >= 0; i--) {
6.                 reverse += string.charAt(i);
7.             }
8.         }
9.     }
10.    return reverse;
11.}
```

18

Coverage Metrics Summary

- Code coverage can help to ensure requirements coverage is complete.
- Basis path testing can identify additional testing paths that may be of interest.
- *Neither* have any guarantees that they will find *all* errors.

How do we find all the issues in the code?

19